FACTORIZARE LU

CROUT

```
function [L U] = crout(A)
  [n n] = size(A);
  L = zeros(n);
  U = eye(n);
  L(1:n,1) = A(1:n,1);

  for p = 1 : n
    for i = p : n
        s = 0;
        for k = 1 : p-1
            s = s+L(i,k)*U(k,p);
        endfor

        L(i,p) = A(i,p)-s;
    endfor

    for j = p+1 : n
        % --------------------------------
        s = 0;
        for k = 1 : p-1
            s = s+L(p,k)*U(k,j);
        endfor

        % echivalent pentru calculul sumei
        % s = L(k,1:k-1)*U(1:k-1,j);

        U(p,j) = (A(p,j)-s)/L(p,p);
    endfor
  endfor
endfunction
```

CHOLESKY

```
function L = cholesky(A)
  [n n] = size(A);
  L = zeros(n);

  for i = 1 : n
    for j = 1 : i-1
      s = 0;
      for k = 1 : j-1
        s = s+L(i,k)*L(j,k);
      endfor
      L(i,j) = (A(i,j)-s)/L(j,j);
    endfor

    s = 0;
    for k = 1 : i-1
      s = s + L(i,k)*L(i,k);
    endfor
    L(i,i) = sqrt(A(i,i)-s);
  endfor
endfunction
```

DOOLITTLE

```
function [L U] = doolittle(A)
  [n n] = size(A);
  L = eye(n);
  U = zeros(n);

  for p = 1 : n
    for j = p : n
        s = 0;
        for k = 1 : p-1
            s = s+L(p,k)*U(k,j);
        endfor
        U(p,j) = A(p,j)-s;
    endfor

    for i = p+1 : n

        s = 0;
        for k = 1 : p-1
            s = s+L(i,k)*U(k,p);
        endfor
        L(i,p) = (A(i,p)-s)/U(p,p);
    endfor
  endfor
endfunction
```

FACTORIZARE QR

GIVENS

```
function [Q R b] = givens(A, b)
  [m n] = size(A);
  Q = eye(m);

  for k = 1 : min(m-1, n)
    for l = k + 1 : m
          r = sqrt(A(k,k)^2 + A(l,k)^2);
          c = A(k,k)/r;
          s = -A(l,k)/r;

          aux = [c -s; s c]*[A(k,k:n); A(l,k:n)];
          A(k,k:n) = aux(1, :);
          A(l,k:n) = aux(2, :);

          aux = [c -s; s c]*[b(k); b(l)];
          b(k) = aux(1);
          b(l) = aux(2);

          aux = [c -s; s c]*[Q(k,1:m); Q(l,1:m)];
          Q(k,1:m) = aux(1, :);
          Q(l,1:m) = aux(2, :);
      end
   end

  Q = Q';
  R = A;
end
```

GRAM-SCHMIDT

```
function [Q, R] = Gram_Schmidt(A)
  [m n] = size(A);
  Q = zeros(m,n);
  R = zeros(n);

  for j = 1 : n
    for i = 1 : j-1
      R(i,j) = Q(:,i)' * A(:,j);
    endfor

    s = zeros(m,1);
    for i = 1 : j-1
      s = s + R(i,j) * Q(:,i);
    endfor
    %%% Echivalent pentru instructiunea
    %%  s = Q(:, 1:j-1) * R(1:j-1, j);

    aux = A(:,j) - s;

    R(j,j) = norm(aux,2);
    Q(:,j) = aux/R(j,j);
  endfor
endfunction
```

HOUSEHOLDER

Algorithm 1 Transformarea unei matrice la forma superior triunghiulară

procedure [Q, R] = ST(A)
    [m,n] = size(A);
    $H = I_n$;
    for $p = 1 : min(m-1, n)$ **do**
        $H_p = I_n$ - $2\frac{v_p v_p^T}{v_p^T v_p}$;
        $A = H_p$ A;
        $H = H_p$ H;
    end for
    $Q = H^T$;
    R = A;
end procedure

METODE ITERATIVE

JACOBI // forma matriceala

```
function [x, err, steps] = Jacobi_matrix_form(A, b, x0, tol, max_steps)
  N = diag(diag(A));
  P = N - A;
  inv_N = inv(N);
  G = inv(N) * P;
  c = inv(N) * b;
  prev_x = x0;
  steps = 0;

  while 1
    x = G * prev_x + c;

    err = norm(x - prev_x);
    if (++steps == max_steps) || (err < tol)
      return;
    endif

    prev_x = x;
  endwhile
endfunction
```

GAUSS-SEIDEL // forma nematriceala

```
function [x succes iter] = GaussSeidel(A, b, x0, tol, maxiter)
  [n n]=size(A);
  succes=0;
  iter=maxiter;
  x=zeros(n,1);

  while maxiter > 0
    maxiter--;

    for i=1:n
      suma=A(i,1:i-1)*x(1:i-1)+A(i,i+1:n)*x0(i+1:n);
      x(i)=(b(i)-suma)/A(i,i);
    endfor

    if norm(x-x0)<tol
      succes=1;
      break;
    endif
    x0=x;
  endwhile
  iter=iter-maxiter;
endfunction
```

ECUATII NELINIARE – SOLUTII

METODA BISECTIEI

```
function [r, err, steps] = bisection(f, a, b, tol, max_steps)
  steps = 0;
  while 1
    r = (a + b) / 2;
    if feval(f, r) * feval(f, a) < 0
      b = r;
    else
      a = r;
    endif

    err = b - a;
    if (++steps == max_steps) || (err < tol)
      return;
    endif
  endwhile
```

VECT SI VAL PROPRII

METODA PUTERII

```
function [lambda, x, err, steps] = power_method(A, x0, tol, max_steps)
  steps = 0;
  prev_x = x0;

  while 1
    x = A * x_prev;
    x = x / norm(x);
    lambda = x' * A * x;

    err = norm(x - prev_x); % abs(lambda - prev_lambda)
    if (++steps == max_steps) || (err < tol)
      return;
    endif

    prev_x = x;
  endwhile
endfunction
```